

Functional Testing

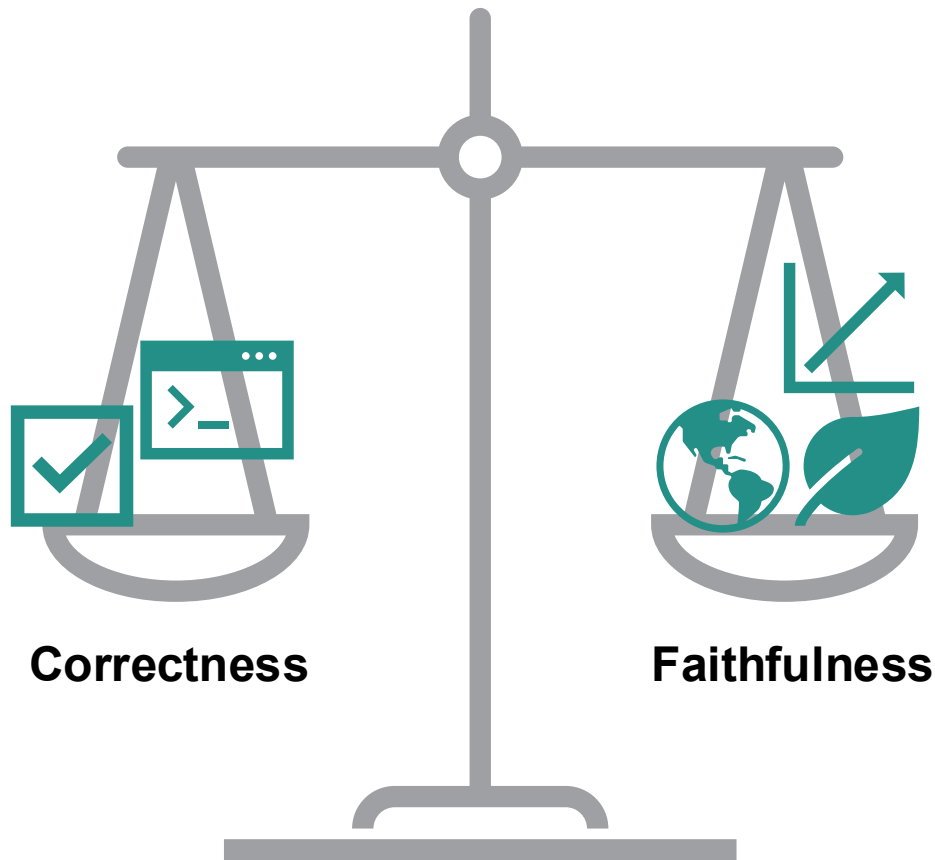
Reasoning about the scientific faithfulness of code

Adrianna Foster

Scientist V, NCAR CGD Terrestrial Sciences Section



2nd Workshop on Correctness and Reproducibility for Earth System Models
Wednesday, November 5, 2025



How can we build confidence that our code not only ***runs***, but ***represents science faithfully***?

Code vs. Science: a refactoring story

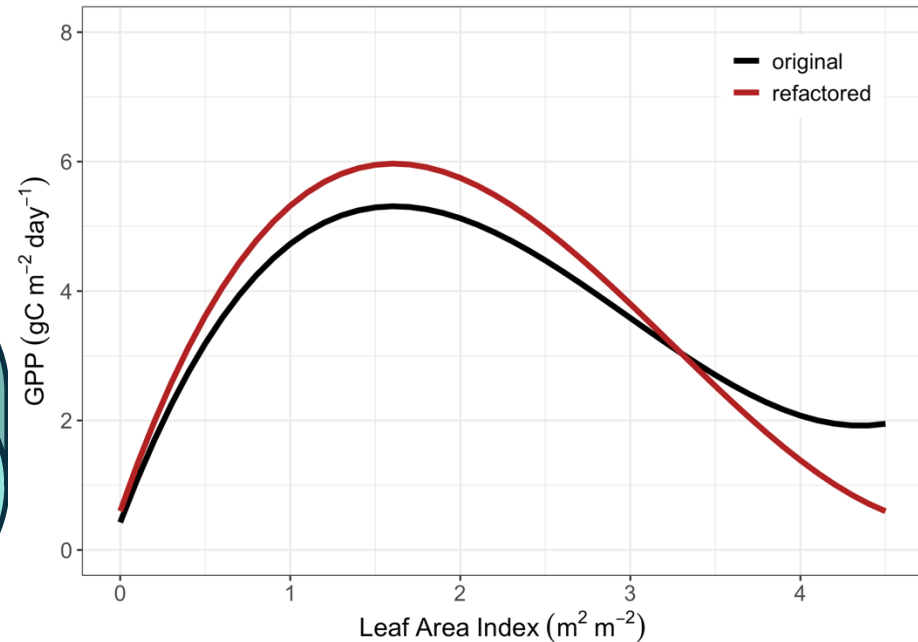
```
cquad = qabs * jmax
call QuadraticRoots(aquad, bquad, cquad, r1, r2)
je = min(r1,r2)

! Initialize intercellular co2
co2_inter_c = init_co2_inter_c

niter = 0
loop_continue = .true.
iter_loop: do while(loop_continue)
! Increment iteration counter. Stop if too many
niter = niter + 1
```

```
! in various search methods such as NR, Secant
if(.not.force_bisection)then
call CiFunc(ci0, &
ft,vcmax,jmax,kp,co2_cpu,co2_mmco2,mm_ko2
can_co2_ppress,can_o2_ppress,can_press,can
gs0,gs1,gs2, &
anet,agross,gs,fval)

! fval = ci_input - ci_predicted
! ci_predicted = ci_input - fval
ci = ci0 - fval
```



Is this a bug or negligible?

Code vs. Science: a refactoring story

```
cquad = qabs * jmax  
call QuadraticRoots(aquad, bquad, cquad, r1, r2)  
je = min(r1, r2)
```

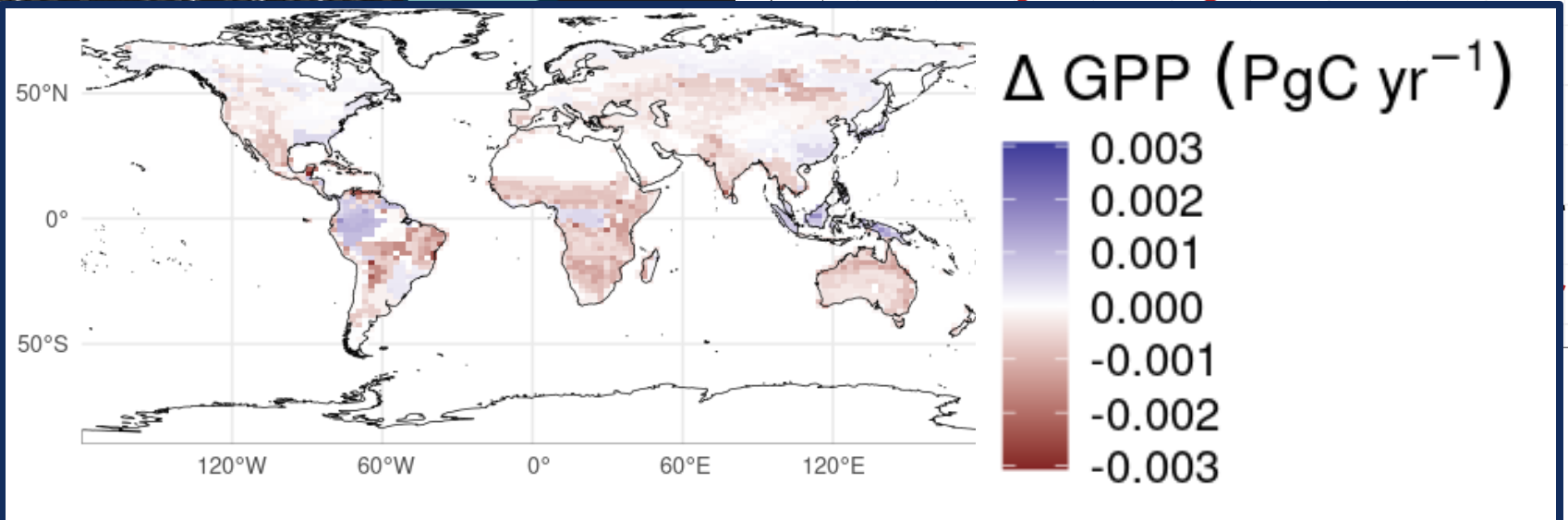
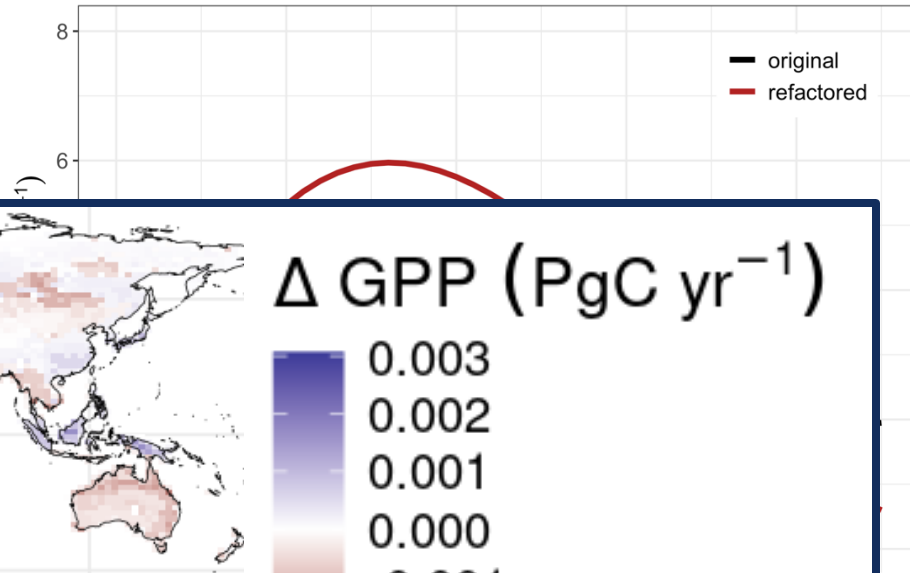
```
! Initialize intercellular co2  
co2_i
```

```
niter  
loop_  
iter_  
!  
ni
```

```
! in v  
if(.nc  
cal
```

```
anet, agross, gs, fval)
```

```
! fval = ci_input - ci_predicted  
! ci_predicted = ci_input - fval  
ci = ci0 - fval
```



Is this a bug or negligible?

Code vs. Science: a refactoring story

```
cquad = qabs * jmax  
call QuadraticRoots(aquad, bquad, cquad, r1, r2)  
je = min(r1, r2)
```

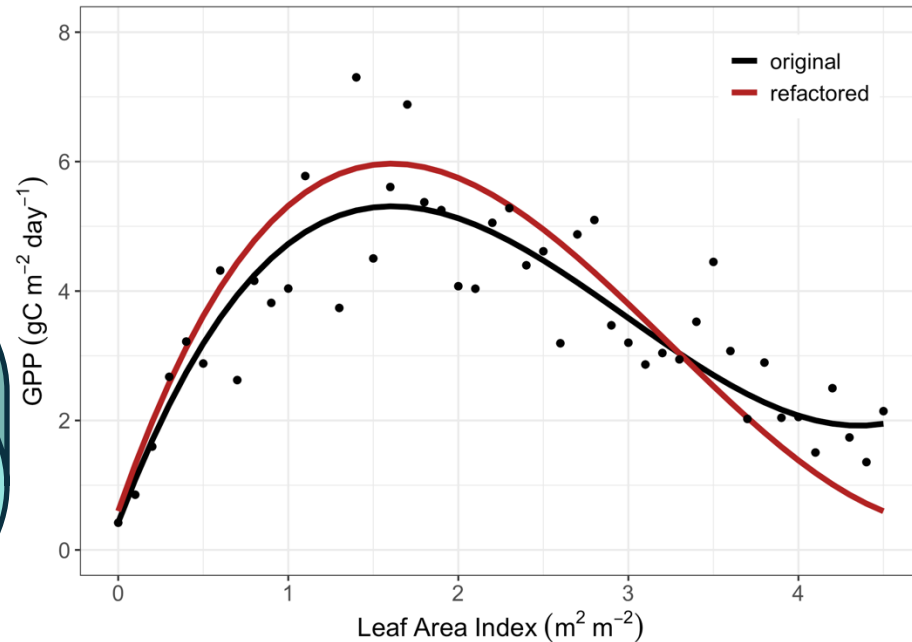
```
! Initialize intercellular co2  
co2_inter_c = init_co2_inter_c
```

```
niter = 0  
loop_continue = .true.  
iter_loop: do while(loop_continue)
```

```
! Increment iteration counter. Stop if too many  
niter = niter + 1
```

```
! in various search methods such as NR, Secant  
if(.not.force_bisection)then  
call CiFunc(ci0, &  
ft, vcmax, jmax, kp, co2_cpu, co2_ppress, can_co2_ppress, can_o2_ppress, can_press, can  
gs0, gs1, gs2, &  
anet, agross, gs, fval)
```

```
! fval = ci_input - ci_predicted  
! ci_predicted = ci_input - fval  
ci = ci0 - fval
```

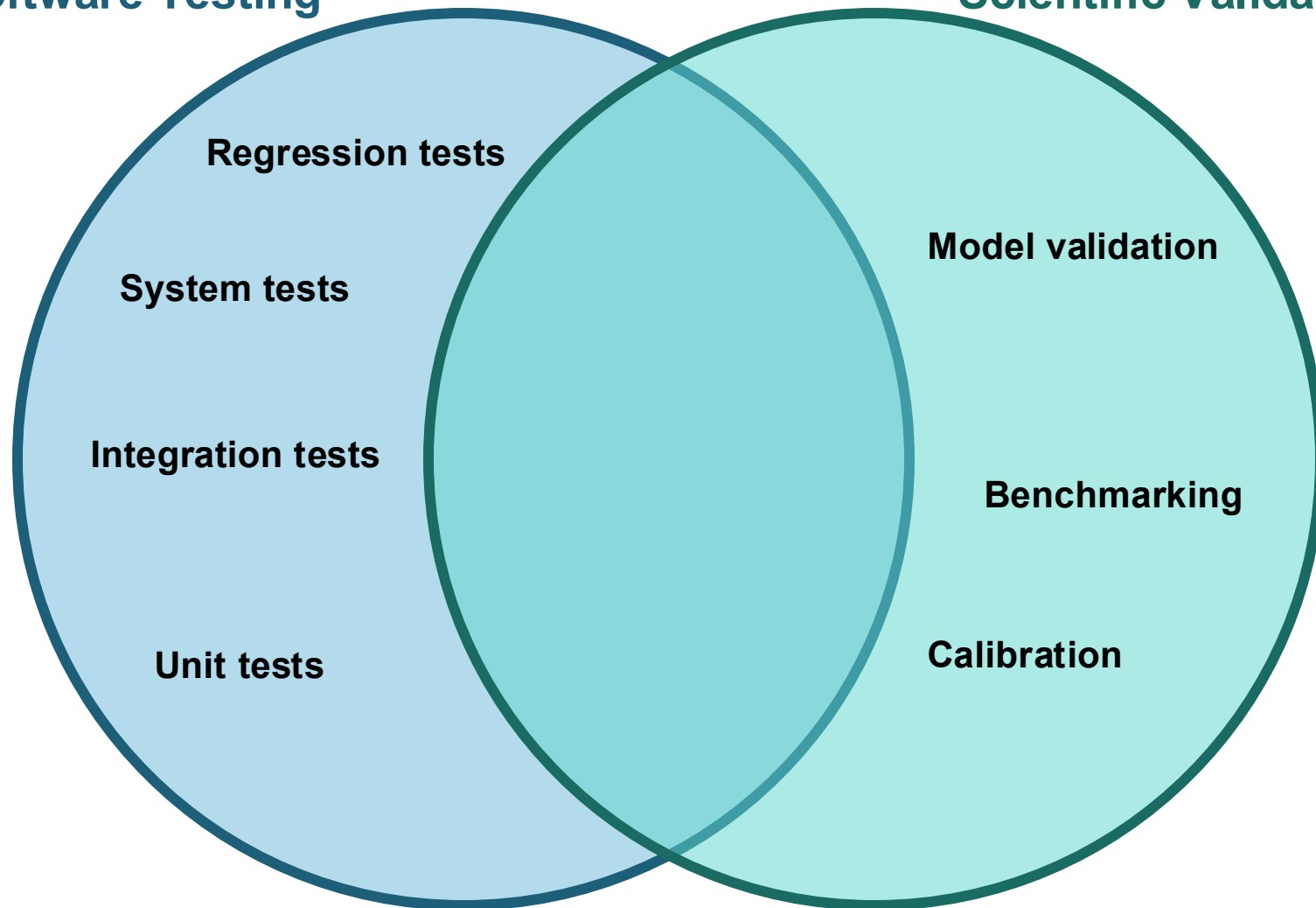


Is this a bug or negligible?

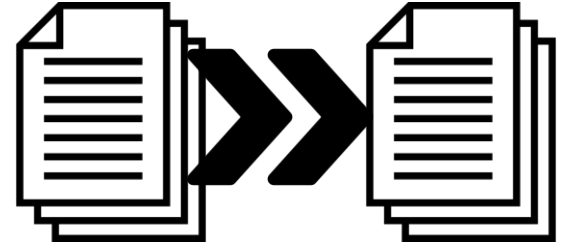
How do we normally test our models?

Software Testing

Scientific Validation



Regression & System Testing



- Ensures model produces consistent results
- Does the model produce the same results (i.e. “bit-for-bit”) when we expect it to?
 - restarts
 - different processors
 - updates that shouldn’t “change answers”

CGD Software Engineers do these **ALL THE TIME**

No science involved – mostly ensures code is working as expected

Unit Testing

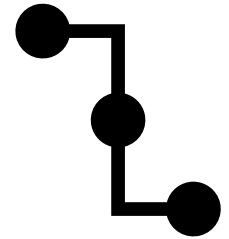


- Tests “units” of software
- Well-defined inputs and outputs
 - checking for errors
 - generally only test one or a few methods at a time
 - must have PASS/FAIL condition

CGD Software Engineers do these **FREQUENTLY** (for infrastructure code)

Still mostly ensuring software is working as expected

Integration Testing

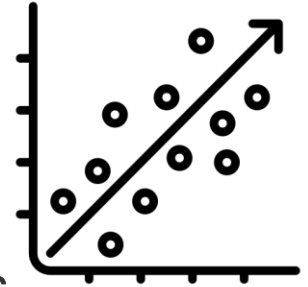


- Tests interactions between components or modules
- Ensures that combined pieces of software behave correctly together
 - checking for errors
 - still usually has PASS/FAIL condition

Generally would do this **SOMETIMES**

Still about *software correctness*, at the system level – verifying that multiple parts can “talk” to each other coherently

Model Validation



- Comparing production-run model output to observations
 - maybe with controlled conditions
- Crucial for verifying a model accurately represents reality
- Model calibration
- Component-specific methods, observations, etc.

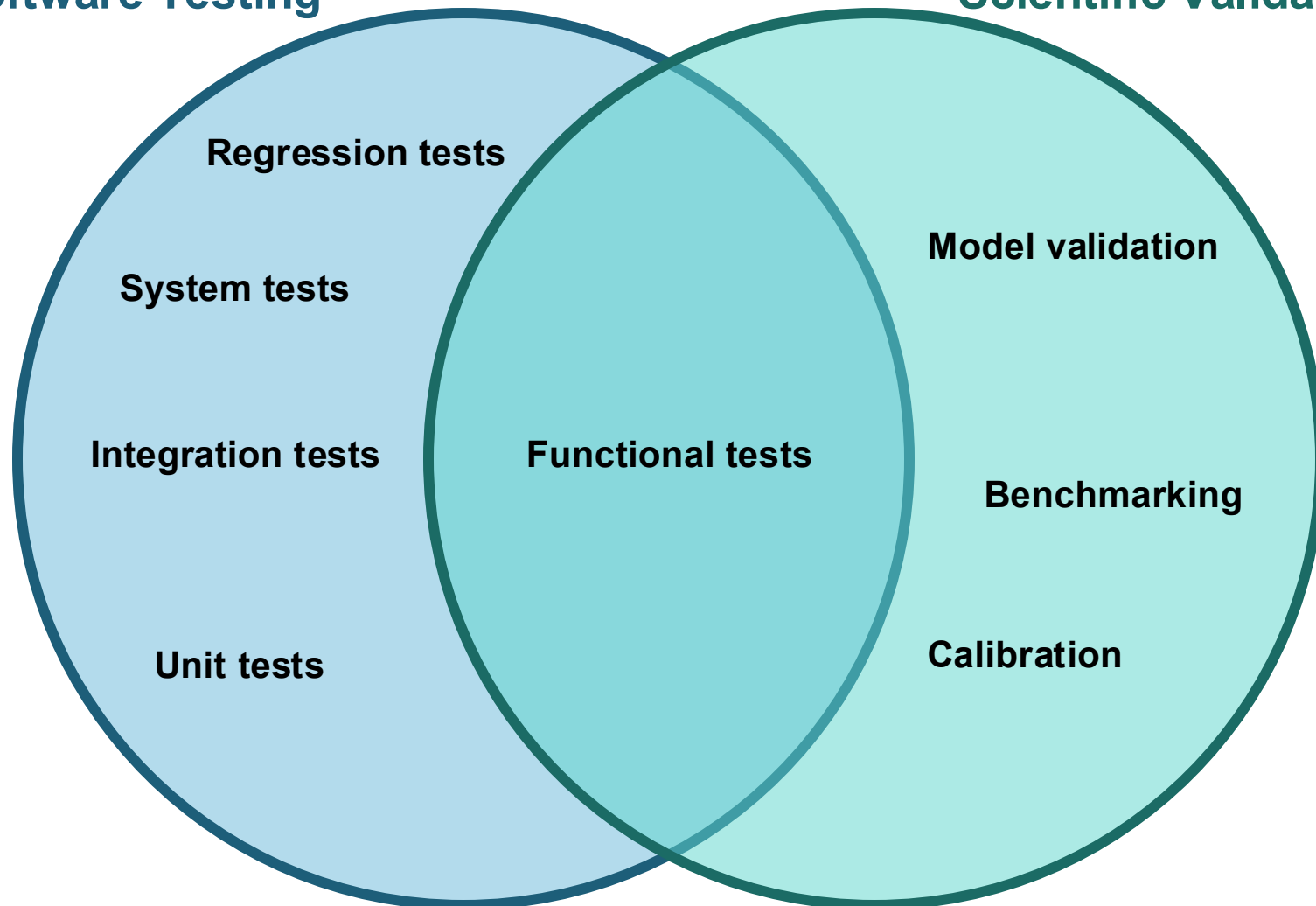
Scientists do this **ALL THE TIME**

Critical yet can be difficult with interactive model processes

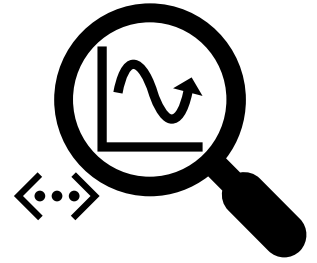
How do we normally test our models?

Software Testing

Scientific Validation



Functional Testing



- Tests small, self-contained pieces of model science or functionality
- Uses production code (not mocks or stubs)
- Focuses on *behavior* and *scientific relationships*, not exact numbers
- Inputs are controlled or idealized to isolate one process
- Compare expected qualitative or quantitative trends

Ensures *scientific faithfulness* – the code still represents the intended science, even as it evolves

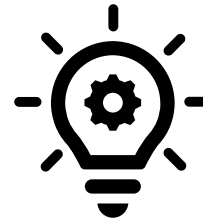
Functional Testing vs. Prototyping



Functional Testing

- Uses **established code**
- Goal: **evaluate scientific behavior**
- Checks for **faithfulness**
- Repeatable, documented, part of testing suite

Testing code's scientific faithfulness

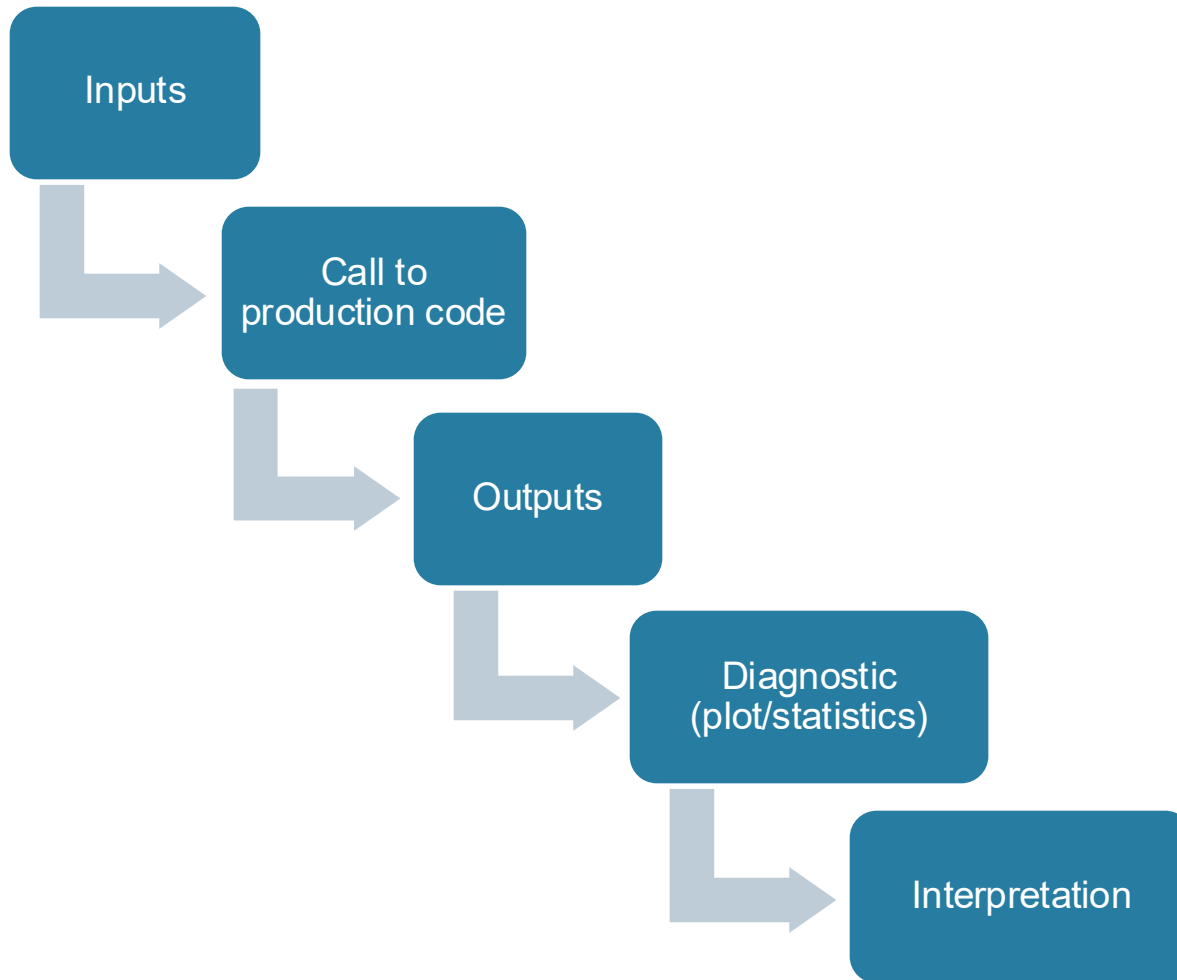


Prototyping

- Uses experimental or evolving code
- Goal: **explore new ideas or formulations**
- Checks for **potential or plausibility**
- Often messy, non-repeatable, transient
- Usually in a **higher-level language**

Discovering/testing new behavior

Anatomy of a Functional Test

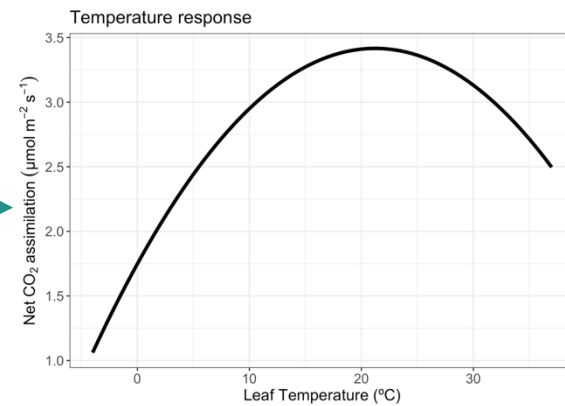
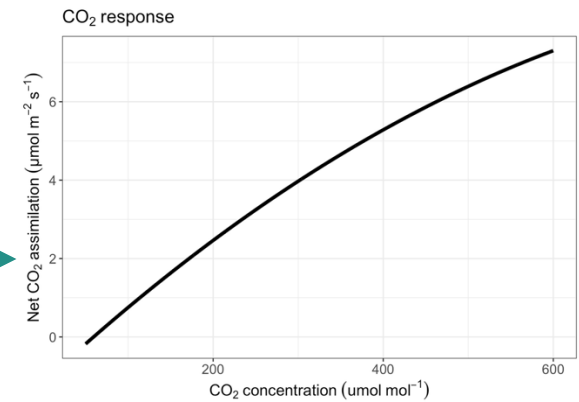
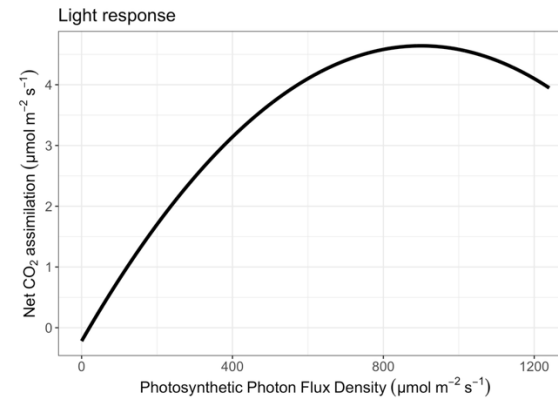


Defining “Faithfulness”

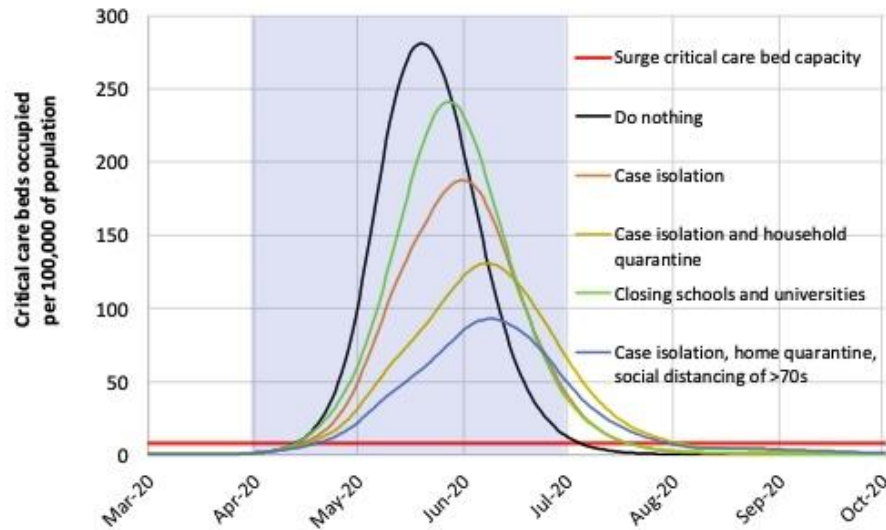
```
subroutine LeafLayerPhotosynthesis( &  
    par_abs,      & ! in  
    ft,           & ! in  
    vcmax,        & ! in  
    jmax,         & ! in  
    kp,           & ! in  
    gs0,          & ! in
```

Defining “Faithfulness”

```
subroutine LeafLayerPhotosynthesis( &  
    par_abs,      & ! in  
    ft,           & ! in  
    vcmax,        & ! in  
    jmax,         & ! in  
    kp,           & ! in  
    gs0,          & ! in
```



Failed faithfulness in science software



CovidSim: epidemiological model for COVID-19

“systematic errors”

“totally unreliable”

“buggy mess”

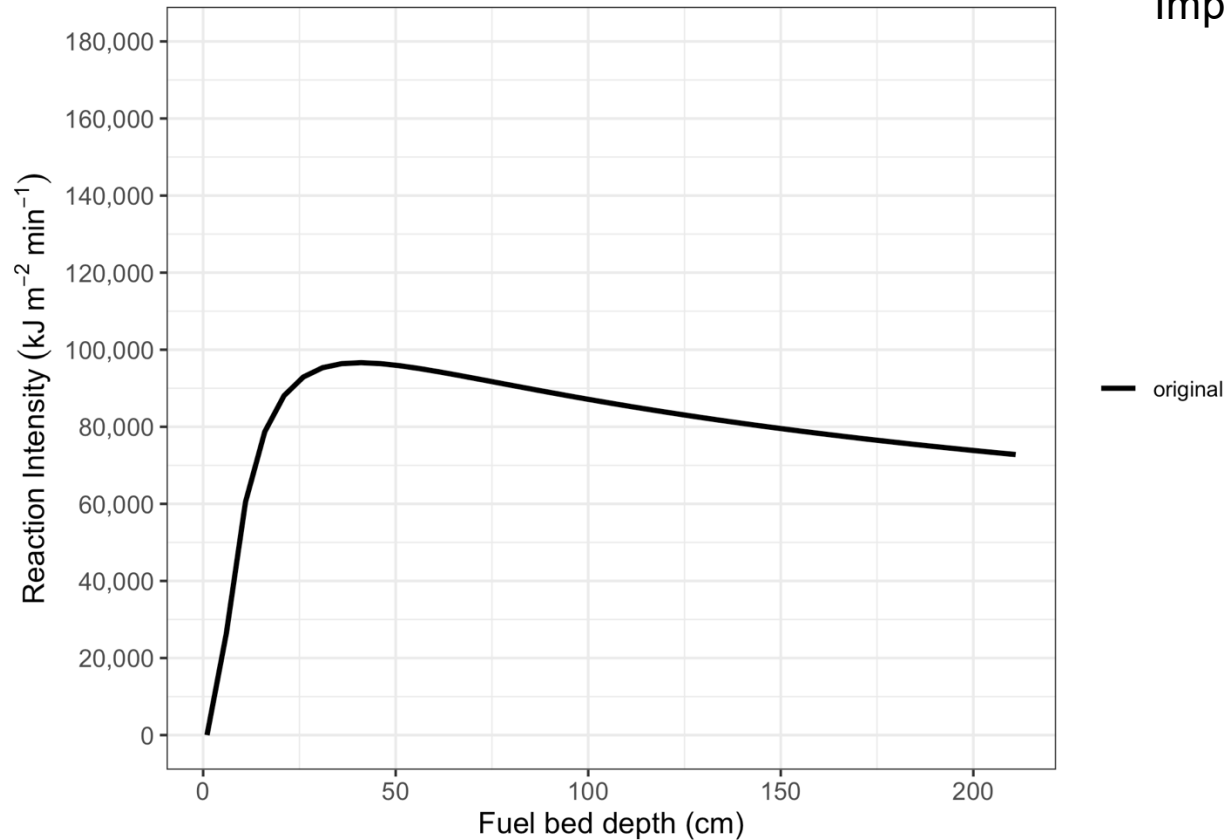
“developed without...ceremony and rigor”

Cleaning up and releasing the code was not a top priority at the time

Though eventually several studies backed up the model and outputs, this reduced faith in epidemiological models

Example: fire spread

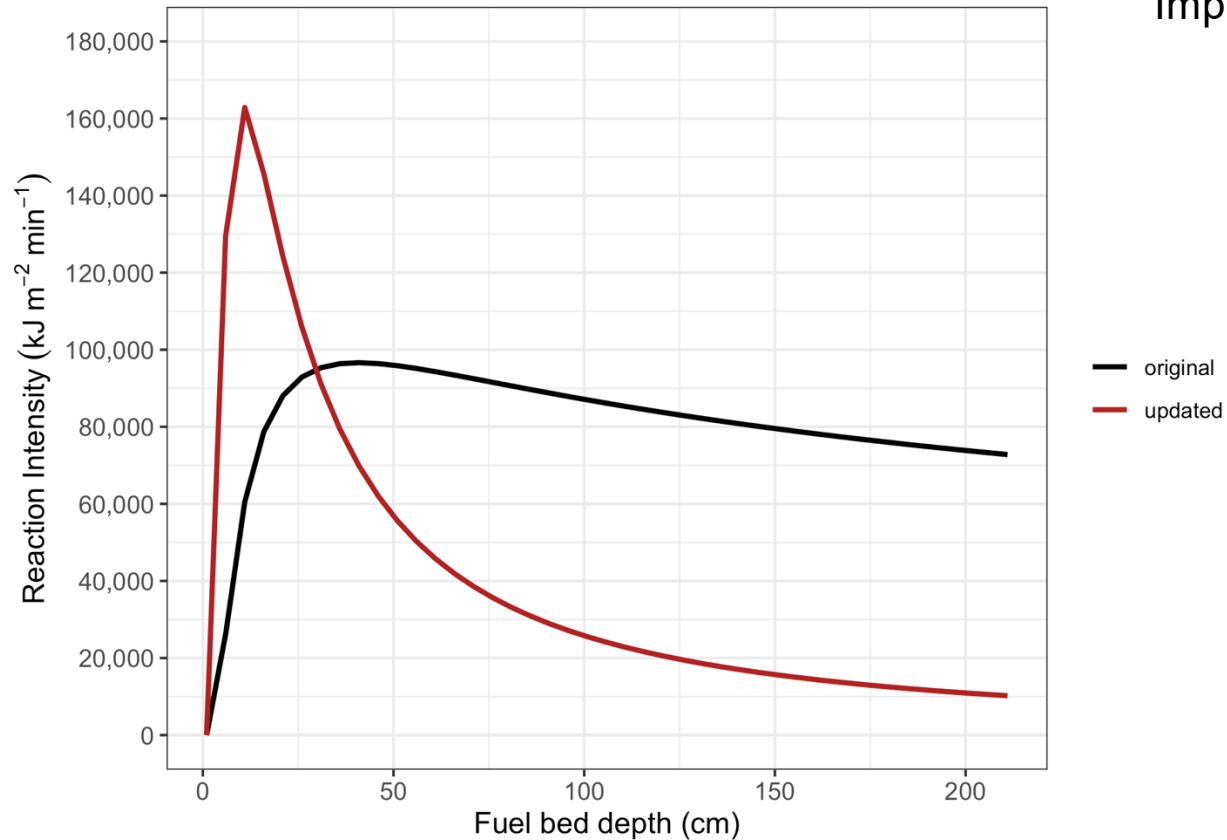
Fuel depth vs. reaction intensity
Important for calculating fire spread



Redrawn from Andrews et al. 2018

Example: fire spread

Fuel depth vs. reaction intensity
Important for calculating fire spread



Redrawn from Andrews et al. 2018

Implementing a functional test

```
You, 2 minutes ago | 1 author (You)
1  program FatesTestR0S
2
3      use FatesConstantsMod,          only :
4      use FatesArgumentUtils,        only :
5      use FatesUnitTestParamReaderMod, only :
6
7      implicit none
8
9      ! LOCALS:
10     type(fates_unit_test_param_reader) :: p
11     character(len=:), allocatable      :: p
12     real(r8),                          :: S
13     real(r8),                          :: b
14     real(r8),                          :: p
15     real(r8),                          :: S
```

Stand-alone program
Compiled to an executable

Implementing a functional test

```
! read in parameter file name and DATM file from command line  
param_file = command_line_arg(1)  
  
! read in parameter file  
call param_reader%Init(param_file)  
call param_reader%RetrieveParameters()  
  
! calculate reaction intensity  
call TestReactionIntensity(fuel_depth, reaction_intensity)
```

Helper methods for parameter reads and individual tests

Implementing a functional test

```
subroutine TestReactionIntensity(fuel_depth, reaction_intensity)
!  
! DESCRIPTION:  
! Calculates reaction intensity at different levels of fuel be  
!  
use FatesConstantsMod, only : r8 => fates_r8  
use SFEquationsMod,    only : ReactionIntensity
```

```
do i = 1, num_fd  
  
  fuel_depth(i) = fd_min + fd_inc*(i-1)  
  
  bulk_density = fuel_loading/fuel_depth(i)  
  beta = bulk_density/part_dens  
  beta_ratio = beta/beta_opt  
  
  reaction_intensity(i) = ReactionIntensity(fuel_loading, SAV,  
  MEF)  
end do
```

**Uses production
code**

Implementing a functional test

```
subroutine WriteROSDData(out_file, reaction_intensity, fuel_de  
    propagating_flux, SAV_values, beta_ratio, reaction_velocity  
    !  
    ! DESCRIPTION:  
    ! writes out data from the test  
    !
```

Write out test results so a higher-level language can interpret

Implementing a functional test

```
def build_tests(build_dir:str, cmake_directory:str,
                verbose:bool=False):
    """Builds the test executables

    Args:
        build_dir (str): build directory
        cmake_directory (str): directory where t
```

```
# run cmake and make
run_cmake(cmake_directory, pfunit_path, netcdf_c_path,
run_make(make_j, clean=clean, verbose=verbose)
```


Implementing a functional test

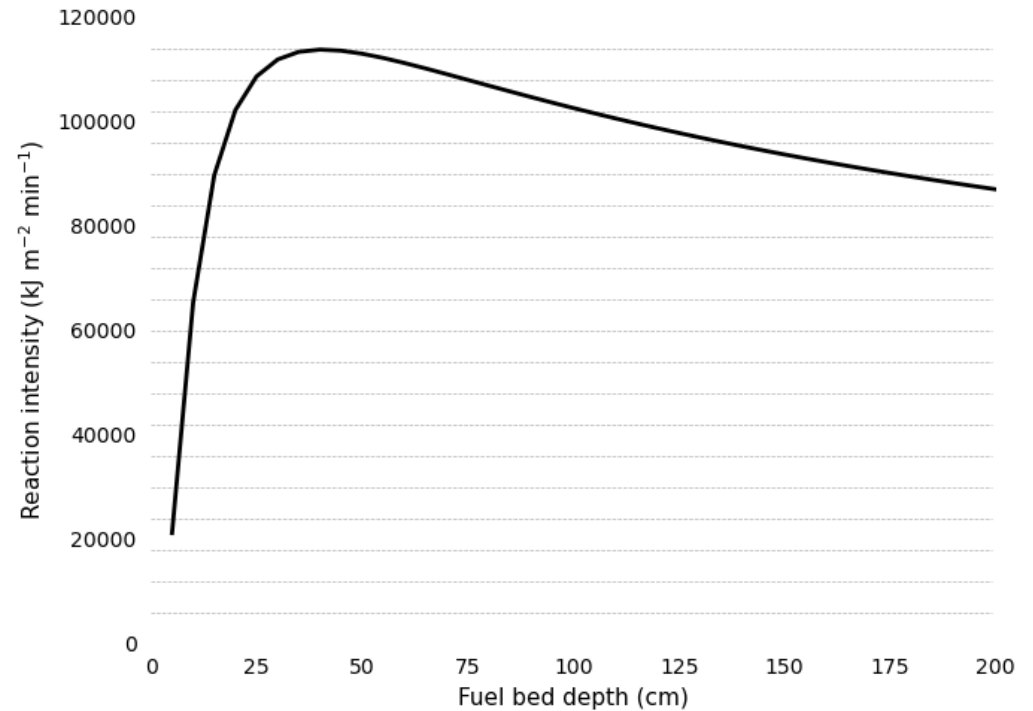
```
# run executables for each test in test list
if run_executables:
    print("Running executables")
    for _, test in test_dict.items():
        # prepend parameter file (if required) to argument list
        args = test.other_args
        if test.use_param_file:
            args.insert(0, param_file)
        # run
        run_fortran_executables(
            build_dir_path, test.test_dir, test.test_exe, run_dir_path, args
        )
```

Implementing a functional test

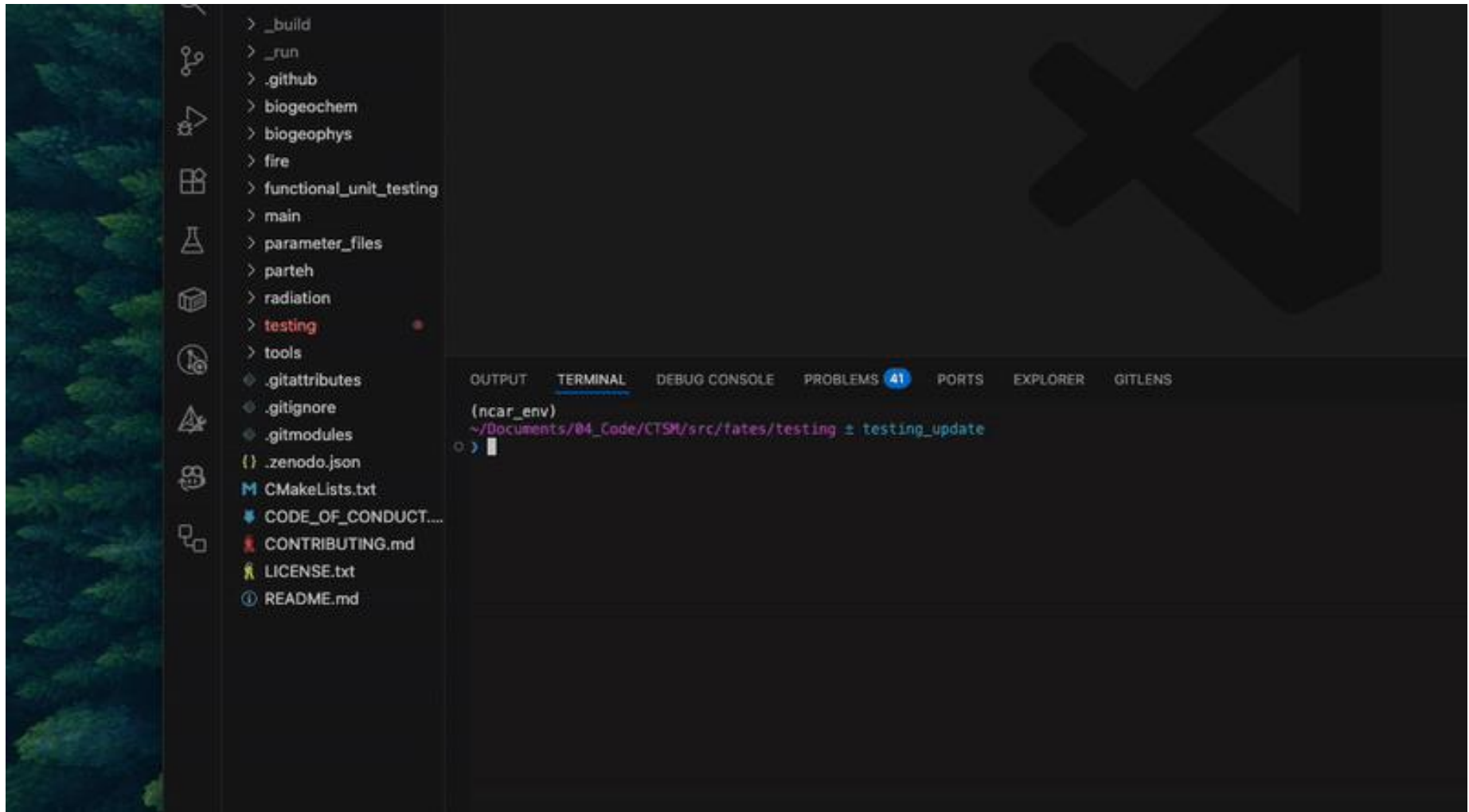
```
@staticmethod
def plot_reaction_intensity(data: xr.
    """Plot reaction intensity

    Args:
        data (xarray DataSet): the da
        save_fig (bool): whether or n
        plot_dir (str): if saving fig

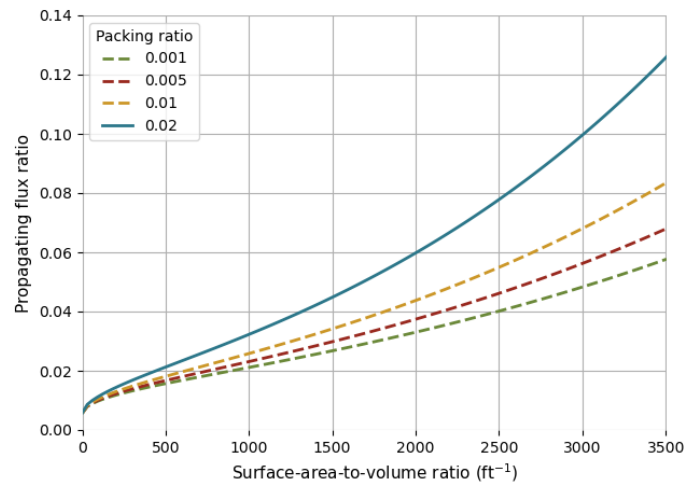
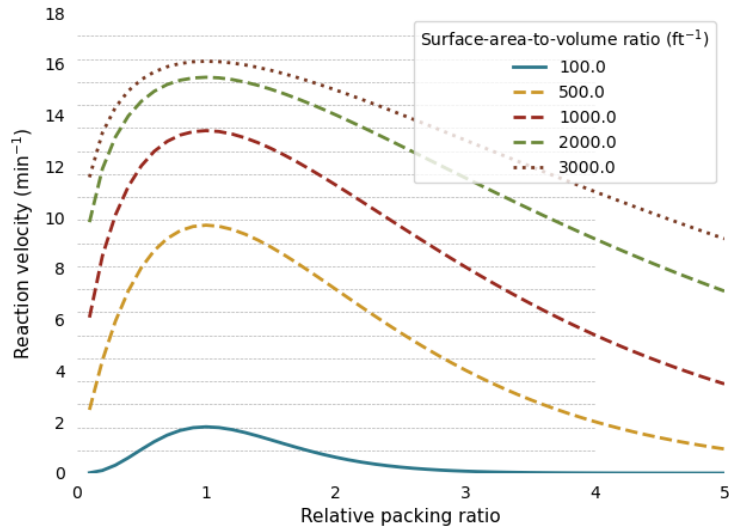
    """
    data_frame = pd.DataFrame(
        {
            "reaction_intensity": dat
            "fuel_depth": data.fuel_d
        }
    )
```



Implementing a functional test



Testing a few fire equations



```
do i = 1, num_beta_r
    beta_ratio(i) = beta_r_min + beta_r_inc*(i-1)

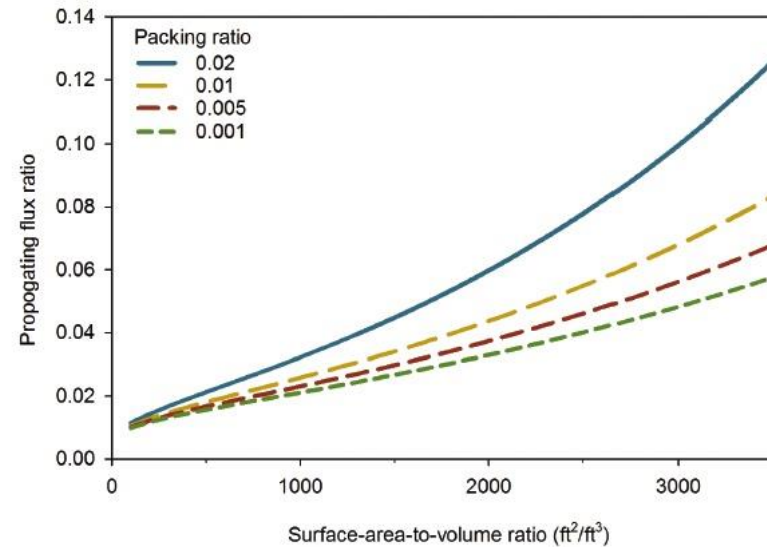
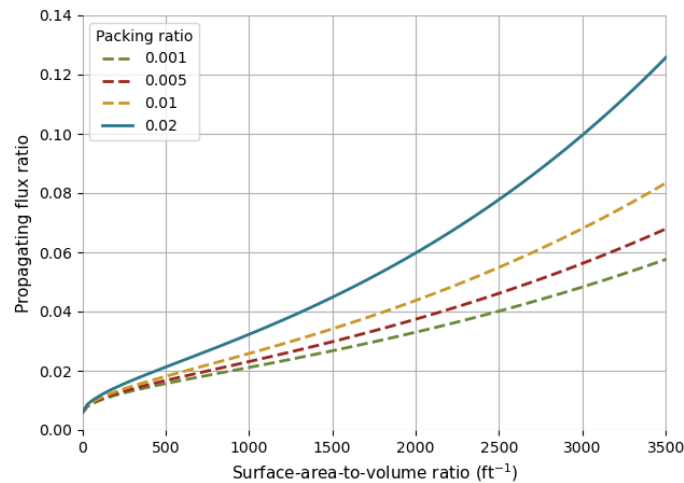
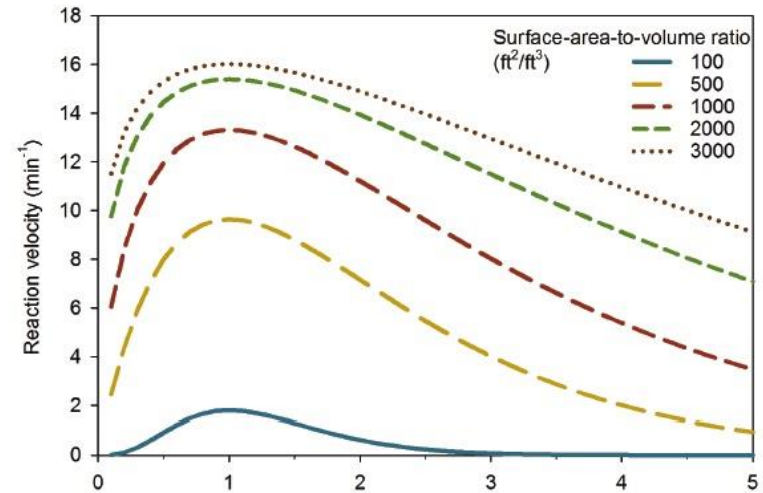
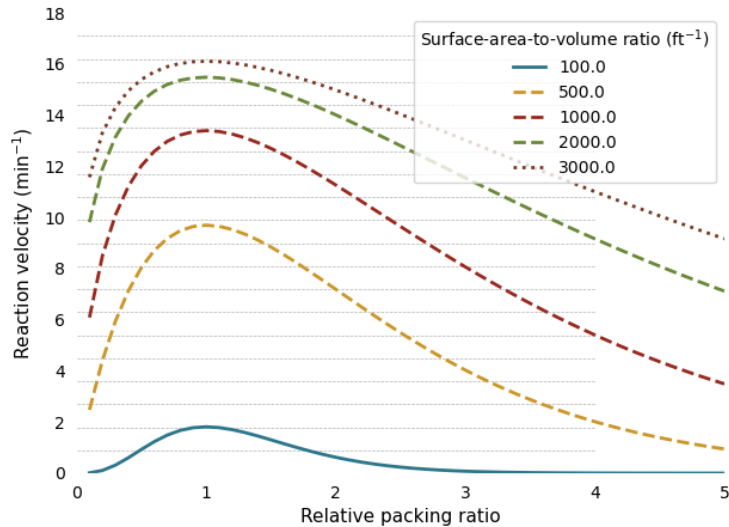
    do j = 1, size(SAV_vals)
        SAV(j) = SAV_vals(j)/30.48_r8 ! convert from /ft to /cm
        max_reaction_vel = MaximumReactionVelocity(SAV(j))
        reaction_velocity(i,j) = OptimumReactionVelocity(max_reaction_vel, SAV(j),
            beta_ratio(i))
    end do
end do
```

```
do i = 1, num_SAV
    SAV(i) = SAV_min + SAV_inc*(i-1)

    do j = 1, size(packing_ratio)
        beta(j) = packing_ratio(j)
        propagating_flux(i,j) = PropagatingFlux(packing_ratio(j), SAV(i))
    end do
end do
```

Testing a few fire equations

Andrews et al. 2018



Testing leaf-level photosynthesis

```
! calculate canopy gas parameters
call GetCanopyGasParameters(can_air_press, can_o2_pp, veg_tempk, can_tempk, &
    can_vpress, veg_esat, leaf_bl_resistance, mm_kco2, mm_ko2, co2_compensation_pt, &
    cf, leaf_bl_conductance, can_vpress_constrained)

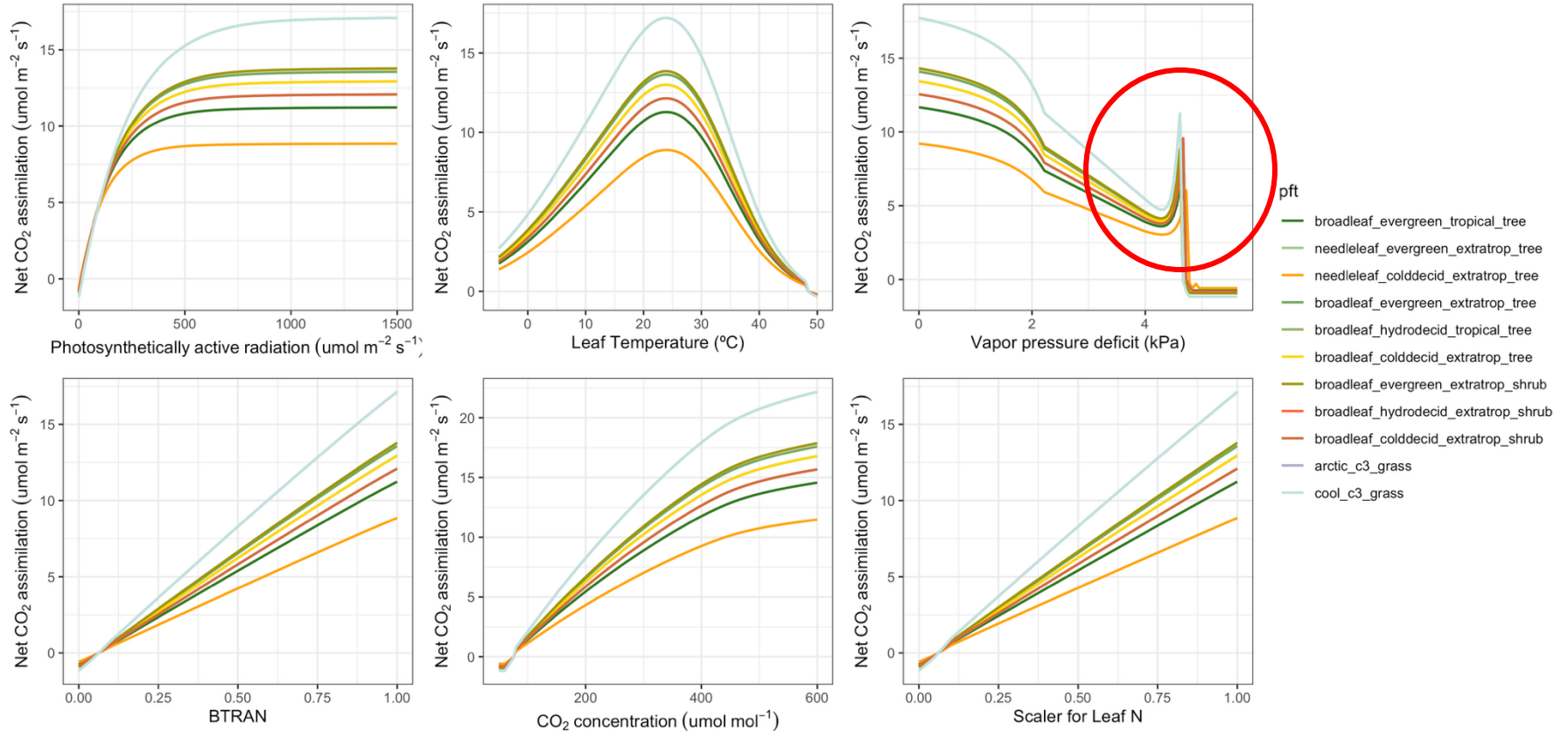
! calculate leaf biophysical rates
call LeafLayerBiophysicalRates(par, ft, EDPftvarcon_inst%vcmax25top(ft,1), &
    param_derived%jmax25top(ft,1), param_derived%kp25top(ft,1), nscler, veg_tempk, &
    dayl_fact, can_tempk, can_tempk, btran, vcmax, jmax, co2_rcurve_islope)

! pulled out from model for now
stomatal_int_btran = max(cf/2.E8_r8, EDPftvarcon_inst%stomatal_intercept(ft)*btran)
    You, 7 months ago • start adding other scalers

! calculate leaf-level photosynthesis
call LeafLayerPhotosynthesis(1.0_r8, par, 0.0_r8, 10.0_r8, 0.0_r8, ft, vcmax, jmax, &
    co2_rcurve_islope, veg_tempk, veg_esat, can_air_press, can_co2_pp, can_o2_pp, btran, &
    stomatal_int_btran, cf, leaf_bl_conductance, can_vpress_constrained, mm_kco2, &
    mm_ko2, co2_compensation_pt, leaf_maintenance_resp, 999.0_r8, leaf_bl_resistance, &
    assimilation, stomatal_resistance, leaf_photo, c13disc, test_out)
```

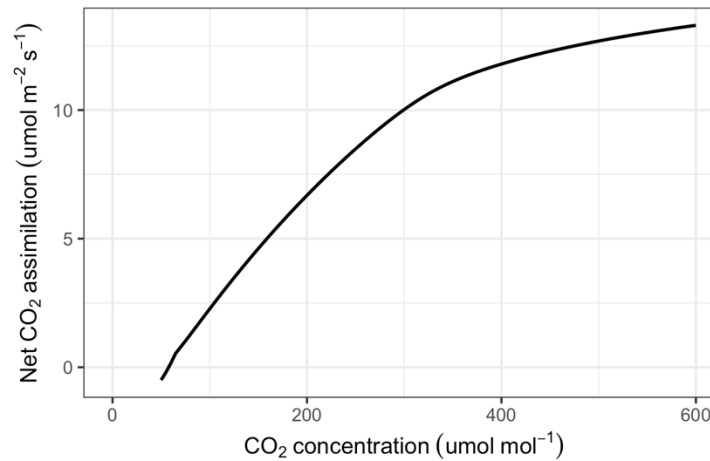
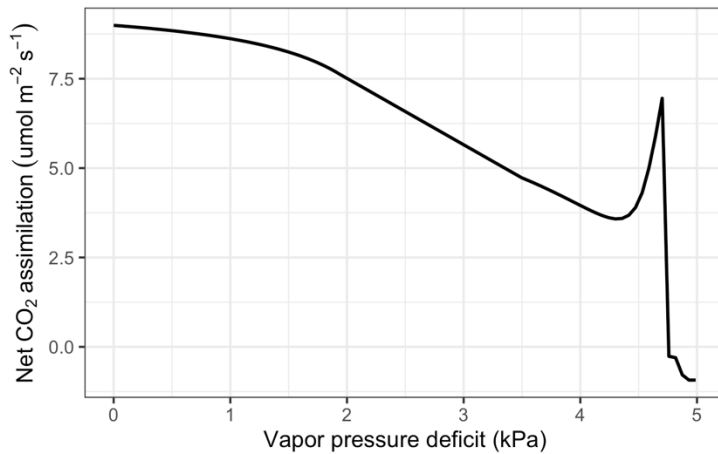
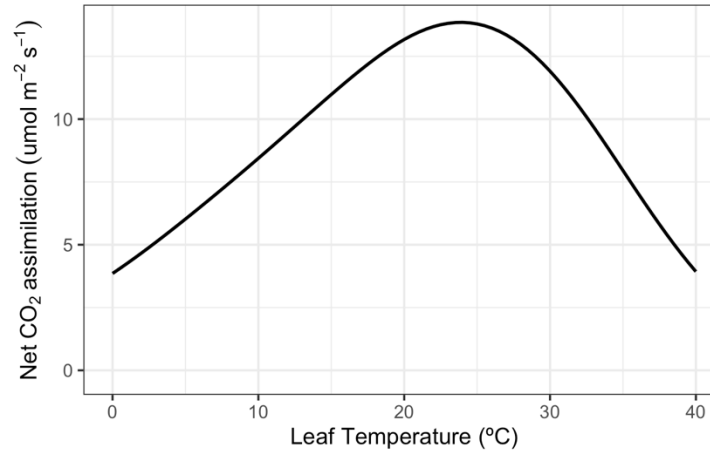
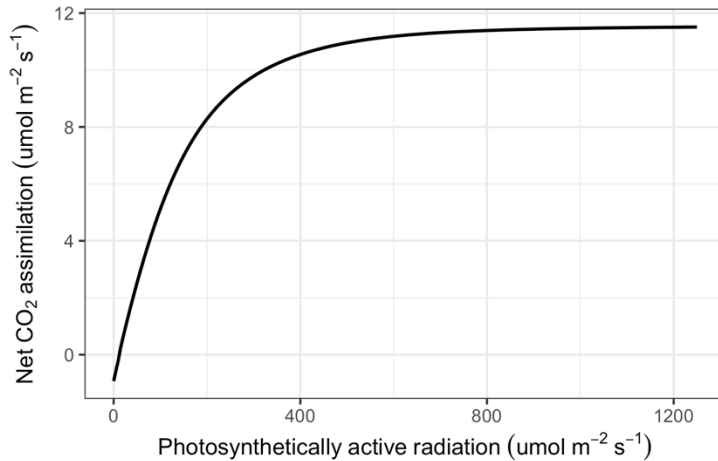
Testing leaf-level photosynthesis

Leaf-level photosynthesis



Testing leaf-level photosynthesis

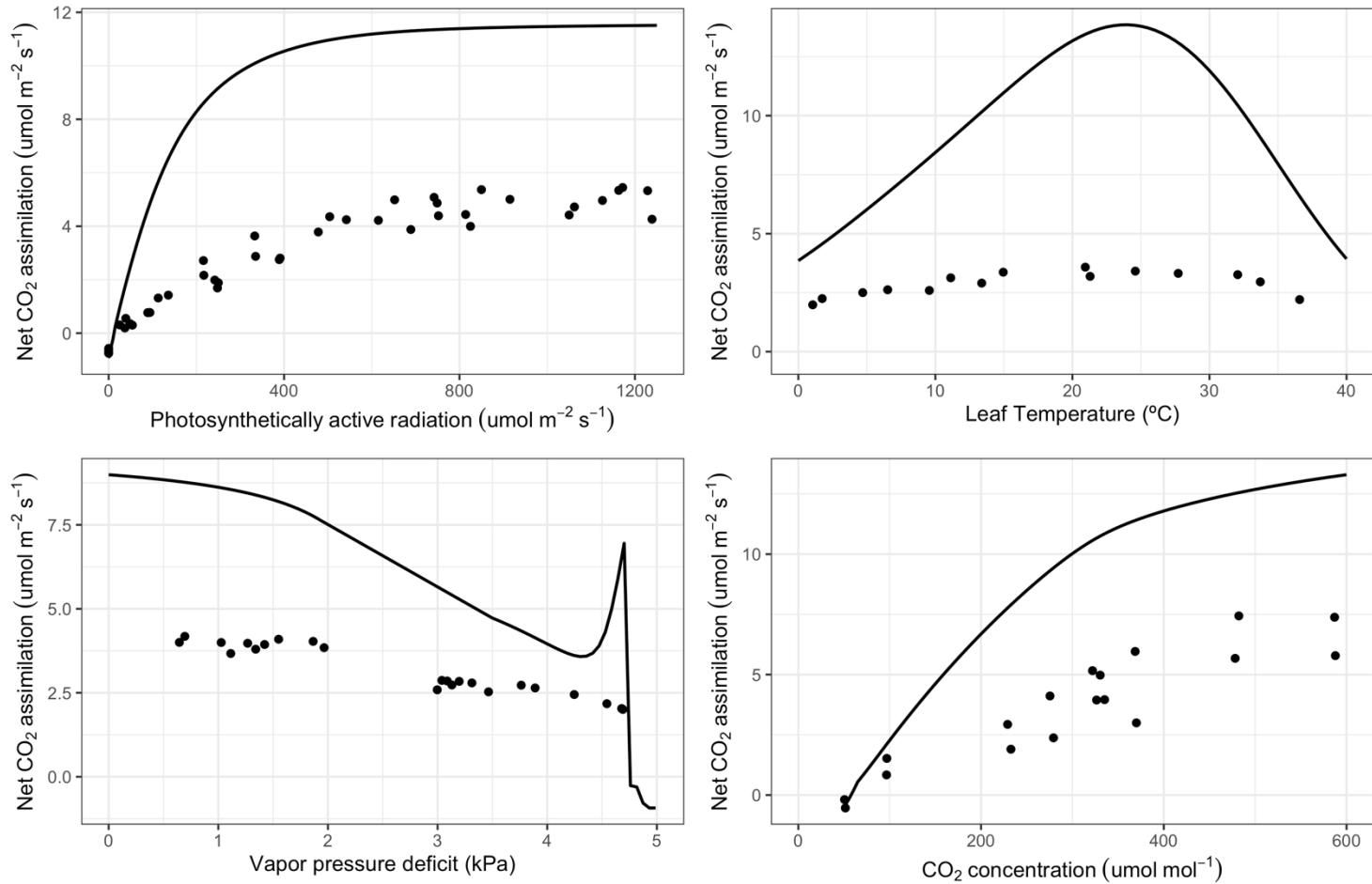
Leaf-level photosynthesis for Jack Pine



Data from BOREAS experiment

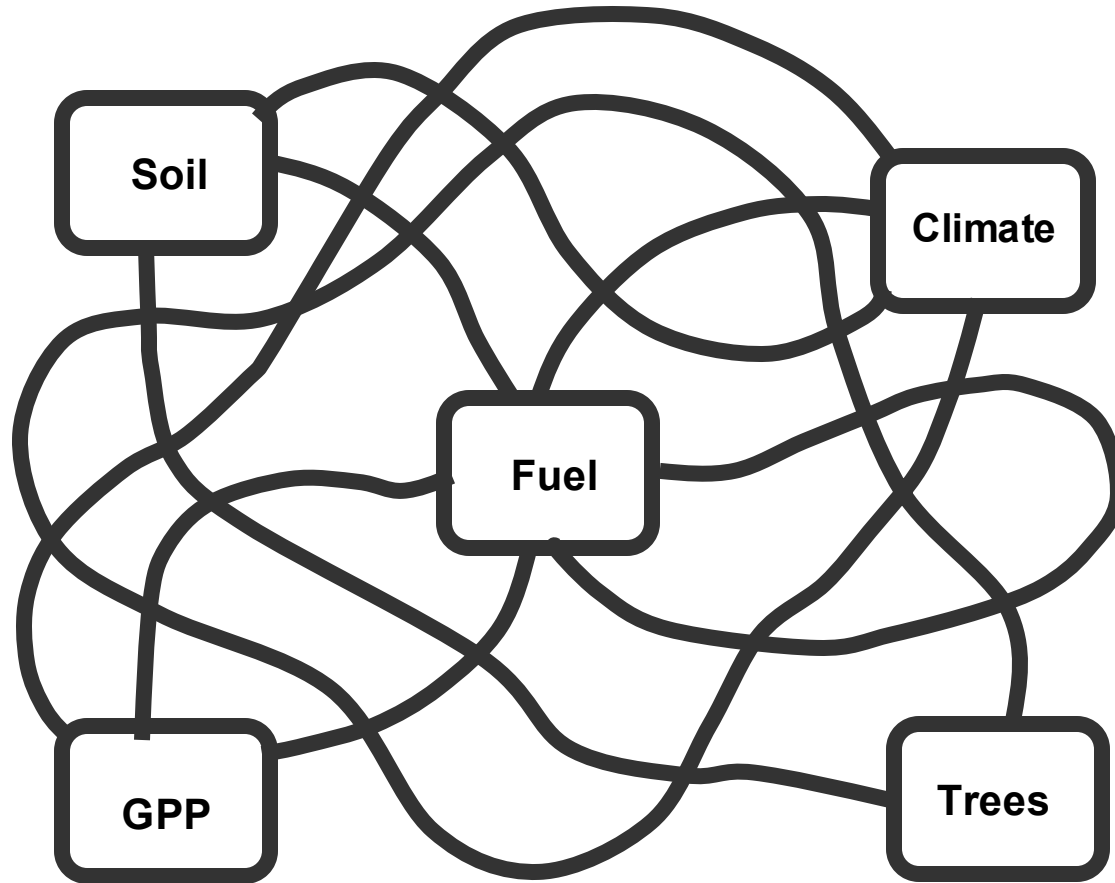
Testing leaf-level photosynthesis

Leaf-level photosynthesis for Jack Pine

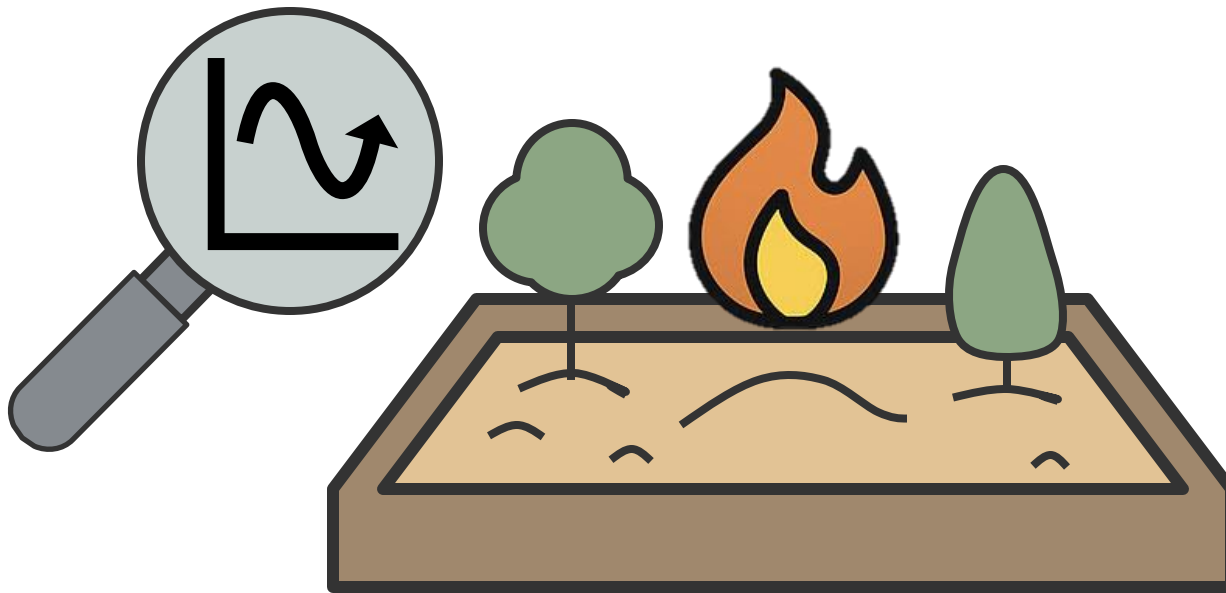


Data from BOREAS experiment

Testing in Isolation



Testing in Isolation



Sandbox for testing

Synthetic fuel types

Standard Fire Behavior Fuel Models: A Comprehensive Set for Use with Rothermel's Surface Fire Spread Model

Joe H. Scott
Robert E. Burgan



183: “moderate load conifer litter”

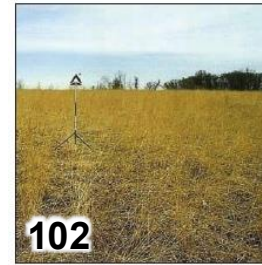
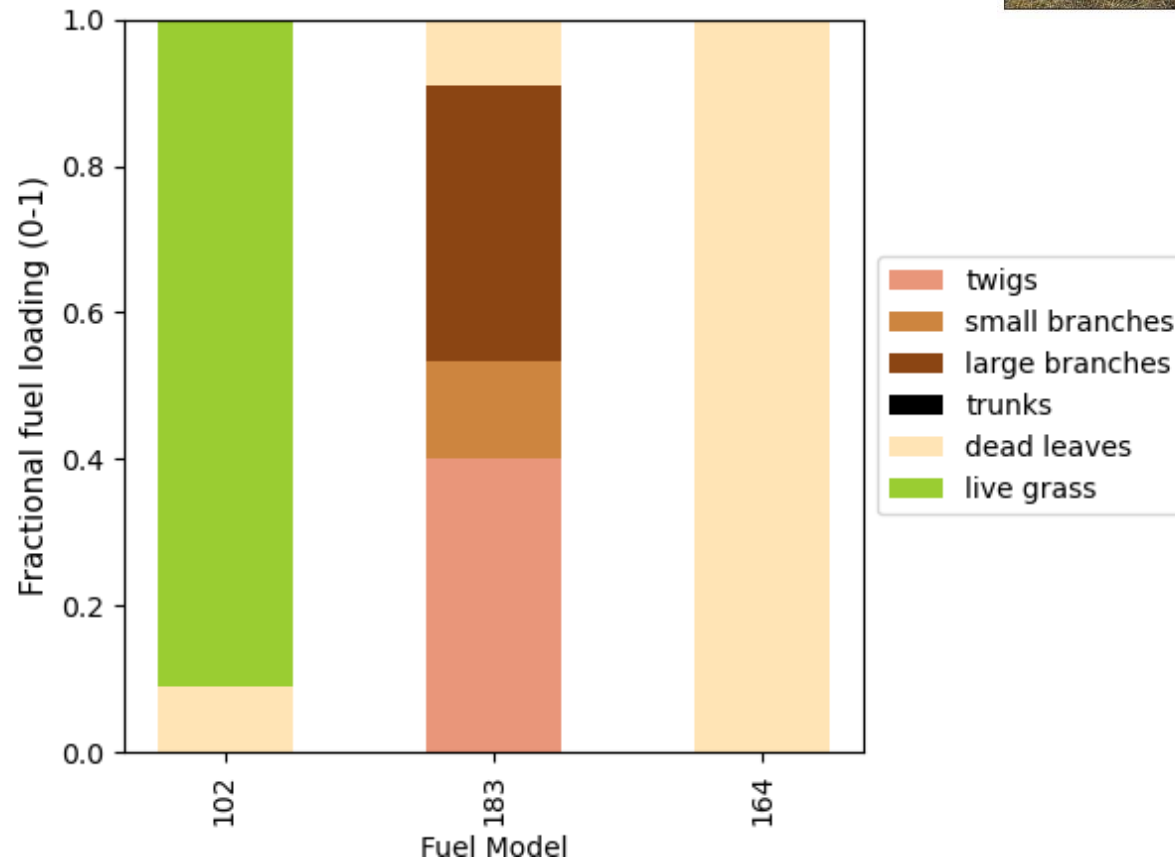


164: “dwarf conifer with understory”

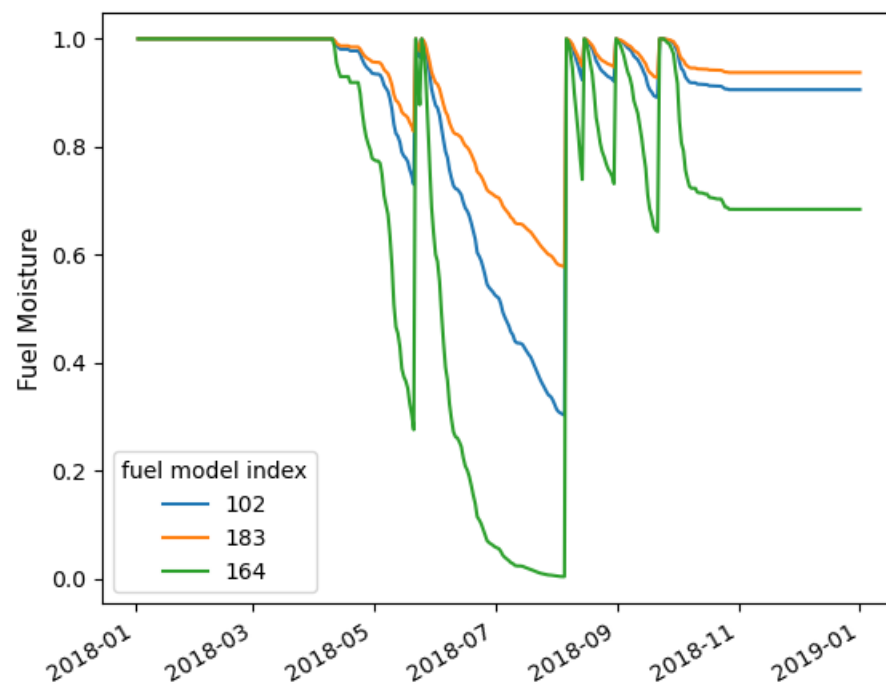
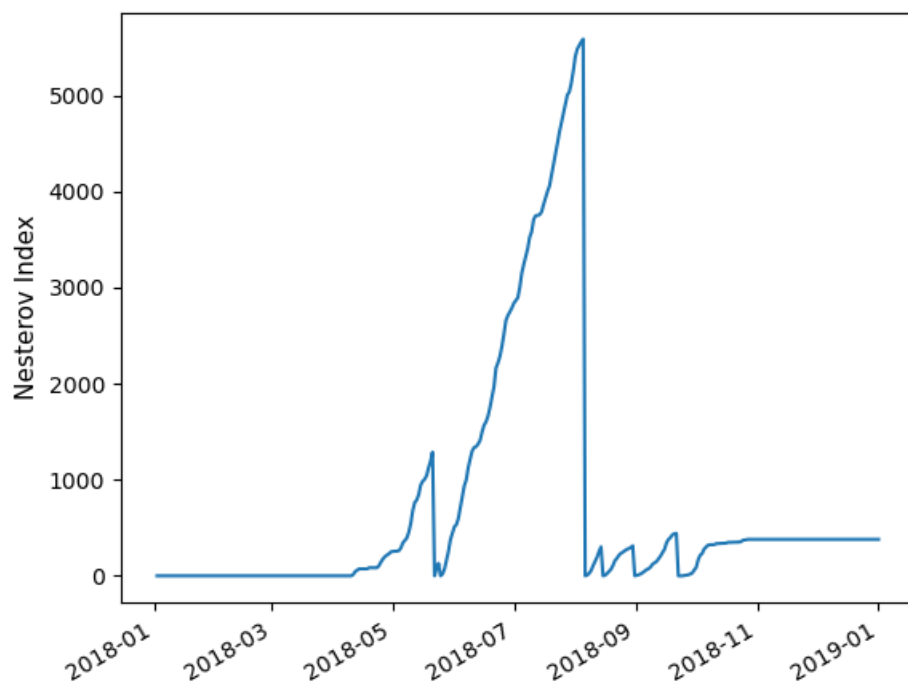
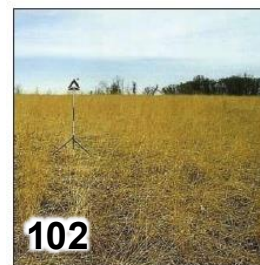


102: “low load dry climate grass”

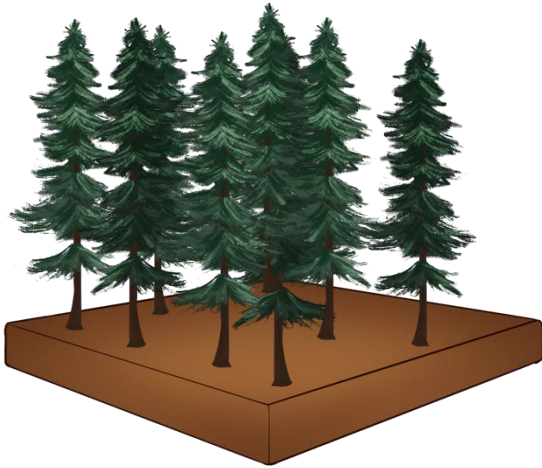
Synthetic fuel types



Testing fuel moisture



Synthetic patches



even-aged evergreen

```
call this%AddPatch(patch_id=2, patch_name='evergreen', area=500.0_r8,  
  ages=(/50.0_r8, 50.0_r8/),  
  dbhs=(/30.0_r8, 25.0_r8/),  
  densities=(/0.015_r8, 0.015_r8/),  
  pft_ids=(/2, 2/),  
  canopy_layers=(/1, 1/))
```



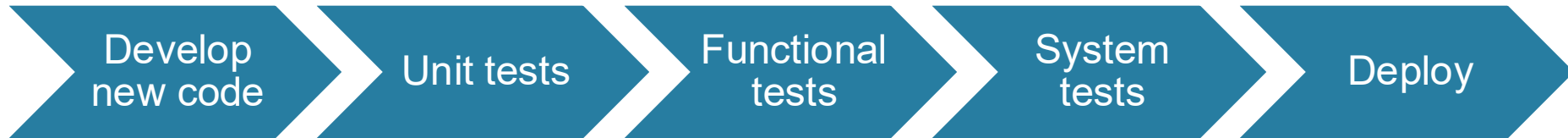
tropical

```
call this%AddPatch(patch_id=1, patch_name='tropical', area=500.0_r8,  
  ages=(/100.0_r8, 80.0_r8, 40.0_r8, 20.0_r8/),  
  dbhs=(/60.0_r8, 50.0_r8, 25.0_r8, 10.0_r8/),  
  densities=(/0.005_r8, 0.008_r8, 0.02_r8, 0.017_r8/),  
  pft_ids=(/1, 1, 1, 1/),  
  canopy_layers=(/1, 1, 2, 2/))
```

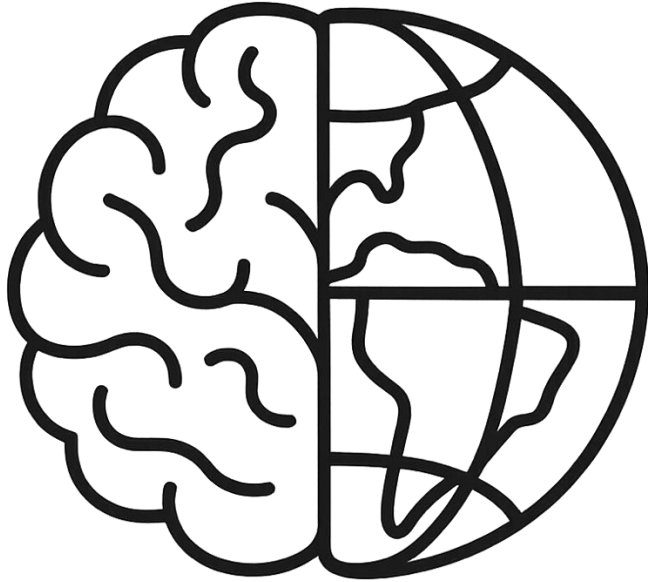
Integrating into workflows



Integrating into workflows



From correctness to confidence



- Care about correctness *and* faithfulness
- Functional testing can formalize trust in model behavior
- As model development proceeds, can ensure these relationships do not change unintentionally

Conclusions

- Functional tests exercise small units of code
- Help understand scientific behavior, not just code correctness
- Small, interpretable, automatable
- Use production code

Acknowledgments

CTSM/FATES SE Team

Erik Kluzek

Sam Levis

Sam Rabin

Greg Lemieux

Ryan Knox

Bill Sacks

Linnia Hawkins

Alper Altuntas



afoster@ucar.edu



Thanks!

Questions?

Functional Testing Practical

https://adrifoster.github.io/functional_testing_tutorial/00_Intro.html

https://github.com/adrifoster/functional_testing_tutorial

Exercise 1.1 Test the propagating flux ratio

Let's start by checking out the source code for this method. It's located in our `FireEquations` class in `src/fire_model/fire_equations.py`.

```
@staticmethod
def propagating_flux(beta: float, sav: float) -> float:
    """Calculates propagating flux ratio [unitless]
    Proportion of reaction intensity that heats adjacent fuel particles to ignition
    Equation A2 in Thonicke et al. 2010 and Eq. 42 Rothermel 1972

    Args:
        beta (float): packing ratio [unitless]
        sav (float): fuel surface area to volume ratio [/cm]
```